ABL PYTHON PROGRAMMING LABORATORY MANUAL

ACTIVITY-BASED LEARNING COURSE CODE & EXECUTION WORKBOOK

Course Title	Activity-Based Learning: Python Programming	**Document Code**	ABL-PY-2026-MANUAL
Target System	Python 3.x Interpreter / IDLE / Jupyter	**Standard Format**	Aim | Procedure | Algorithm | Pseudo Code | Flowchart | Code | Output | Conclusion

EXPERIMENT 01: Basic Input/Output & Arithmetic Operations — LAB SEC: ABL-01

1. AIM

To write and execute a Python program that reads two numeric values from the user and computes their sum, difference, product, division, and modulo remainder.

2. PROCEDURE

1. Open the Python IDLE environment or a code editor and create a new file named `arithmetic_ops.py`.
2. Prompt the user to input two numbers using the `input()` function and parse them into floating-point numbers using `float()`.
3. Perform addition, subtraction, and multiplication on the variables.
4. Check if the second number is non-zero before attempting division and modulo operations to prevent divide-by-zero runtime exceptions.
5. Format and output the computed results cleanly using formatted string literals (f-strings).
6. Save the program and execute it in the Python interactive shell.

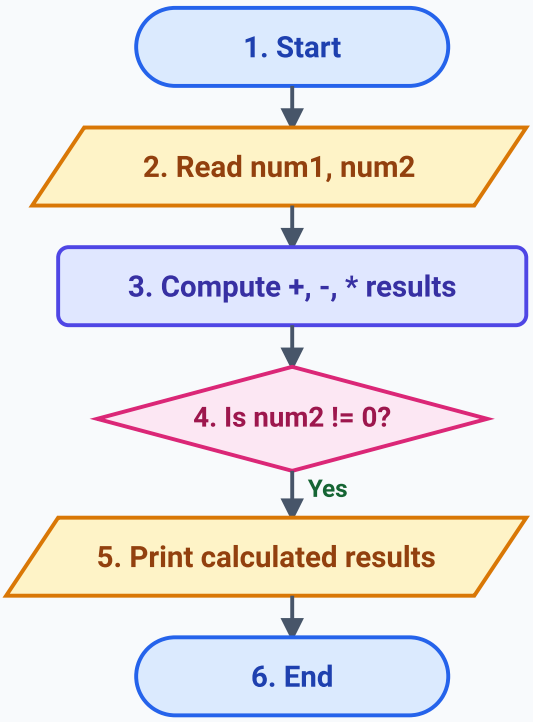
3. ALGORITHM

1. **Step 1:** Start the program execution.
2. **Step 2:** Input variable num1 and num2 from user.
3. **Step 3:** Compute `sum = num1 + num2`.
4. **Step 4:** Compute `diff = num1 - num2`.
5. **Step 5:** Compute `prod = num1 * num2`.
6. **Step 6:** If `num2 != 0`, calculate `div = num1 / num2` and `mod = num1 % num2`; otherwise display warning.
7. **Step 7:** Print the calculated values for all operations.
8. **Step 8:** Stop/End.

4. PSEUDO CODE

```
BEGIN
   READ num1, num2 AS FLOAT
   COMPUTE add_val = num1 + num2
   COMPUTE sub_val = num1 - num2
   COMPUTE mul_val = num1 * num2
   IF num2 != 0 THEN
      COMPUTE div_val = num1 / num2
      COMPUTE mod_val = num1 MOD num2
      PRINT add_val, sub_val, mul_val, div_val, mod_val
   ELSE
      PRINT "Division by zero invalid"
   END IF
END
```

5. FLOWCHART



6. PROGRAM CODE

```
# Experiment 01: Arithmetic Operations in Python
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(f"Addition: {num1} + {num2} = {num1 + num2}")
print(f"Subtraction: {num1} - {num2} = {num1 - num2}")
print(f"Multiplication: {num1} * {num2} = {num1 * num2}")

if num2 != 0:
    print(f"Division: {num1} / {num2} = {num1 / num2:.2f}")
    print(f"Modulus: {num1} % {num2} = {num1 % num2}")
else:
    print("Error: Division by zero is undefined.")
```

7. OUTPUT

```
Enter first number: 24
Enter second number: 5
Addition: 24.0 + 5.0 = 29.0
Subtraction: 24.0 - 5.0 = 19.0
Multiplication: 24.0 * 5.0 = 120.0
Division: 24.0 / 5.0 = 4.80
Modulus: 24.0 % 5.0 = 4.0
```

8. CONCLUSION

The program successfully performs arithmetic operations based on user input while properly taking into account divide-by-zero logic.

EXPERIMENT 02: Prime Number Determination — LAB SEC: ABL-02

1. AIM

To design a Python program to determine whether a user-given positive integer is a Prime number.

2. PROCEDURE

1. Prompt the user for a single integer value.
2. Check whether the input number is less than or equal to 1; if so, identify it immediately as non-prime.
3. Construct a loop starting from 2 up to the square root of the target integer.
4. In each iteration, test if the integer divides evenly by checking if `n % i == 0`.
5. If any factor is discovered, flag the number as non-prime and break the loop.
6. Display the result based on the boolean flag value.

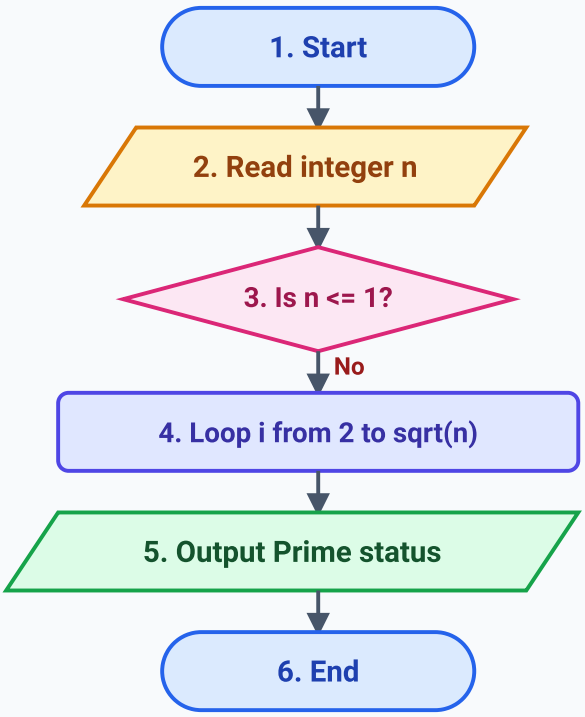
3. ALGORITHM

1. **Step 1:** Start program.
2. **Step 2:** Input integer n.
3. **Step 3:** If `n <= 1`, set `is_prime = False`.
4. **Step 4:** Else loop `i` from 2 to `sqrt(n)`.
5. **Step 5:** If `n % i == 0`, set `is_prime = False` and exit loop.
6. **Step 6:** If `is_prime` is True, print "Prime Number", else print "Not a Prime Number".
7. **Step 7:** Stop.

4. PSEUDO CODE

```
BEGIN
   READ n AS INTEGER
   SET is_prime = TRUE
   IF n <= 1 THEN
      is_prime = FALSE
   ELSE
      FOR i = 2 TO SQRT(n) DO
         IF n MOD i == 0 THEN
            is_prime = FALSE
            BREAK
         END IF
      END FOR
   END IF
   IF is_prime == TRUE THEN
      PRINT n, "is a Prime Number"
   ELSE
      PRINT n, "is Not a Prime Number"
   END IF
END
```

5. FLOWCHART



6. PROGRAM CODE

```
import math

num = int(input("Enter a positive integer: "))
is_prime = True

if num <= 1:
    is_prime = False
else:
    for i in range(2, int(math.isqrt(num)) + 1):
        if num % i == 0:
            is_prime = False
            break

if is_prime:
    print(f"{num} is a Prime Number.")
else:
    print(f"{num} is NOT a Prime Number.")
```

7. OUTPUT

```
Enter a positive integer: 29
29 is a Prime Number.
```

8. CONCLUSION

Prime verification logic was successfully implemented using optimized `O(sqrt(N))` loop techniques.

EXPERIMENT 03: Fibonacci Sequence Generation — LAB SEC: ABL-03

1. AIM

To construct a Python program to generate and print the Fibonacci sequence up to n terms using loop operations.

2. PROCEDURE

1. Read the number of required terms n from the user.
2. Initialize the first two terms of the sequence, `a = 0` and `b = 1`.
3. Execute a loop running n times.
4. In each iteration, print current term a, compute the next term `c = a + b`, and update variables (`a = b`, `b = c`).
5. Ensure proper handling of 0 or negative terms.

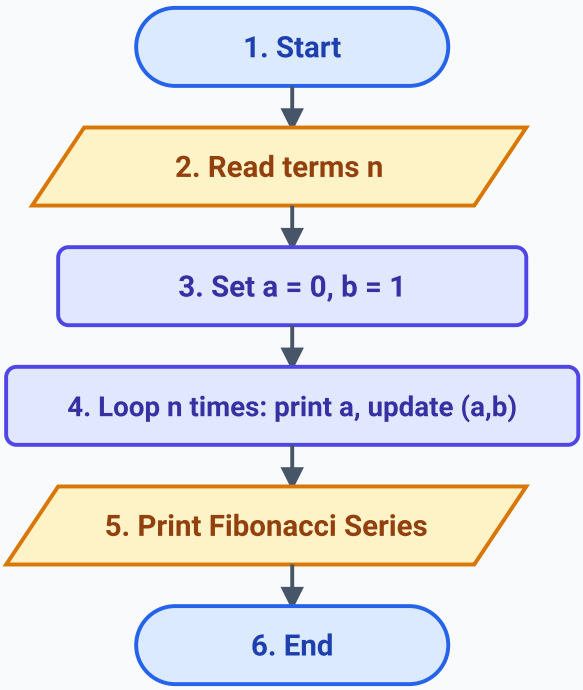
3. ALGORITHM

1. **Step 1:** Start program.
2. **Step 2:** Input total terms n.
3. **Step 3:** Initialize `a = 0`, `b = 1`.
4. **Step 4:** Repeat for `count = 0` to `n - 1`.
5. **Step 5:** Output term a.
6. **Step 6:** Compute `next_term = a + b`, set `a = b`, `b = next_term`.
7. **Step 7:** Stop when iterations complete.

4. PSEUDO CODE

```
BEGIN
   READ n
   SET a = 0, b = 1
   IF n <= 0 THEN
      PRINT "Enter positive count"
   ELSE
      FOR i = 1 TO n DO
         PRINT a
         SET next_val = a + b
         SET a = b
         SET b = next_val
      END FOR
   END IF
END
```

5. FLOWCHART



6. PROGRAM CODE

```
terms = int(input("Enter number of terms: "))
a, b = 0, 1
count = 0

if terms <= 0:
    print("Please enter a positive integer.")
elif terms == 1:
    print(f"Fibonacci sequence: {a}")
else:
    print("Fibonacci sequence:")
    while count < terms:
        print(a, end=" ")
        nth = a + b
        a = b
        b = nth
        count += 1
    print()
```

7. OUTPUT

```
Enter number of terms: 7
Fibonacci sequence:
0 1 1 2 3 5 8
```

8. CONCLUSION

Fibonacci series creation was successfully verified using non-recursive iterative loops.

EXPERIMENT 04: List Operations (Sum, Avg, Search & Sort) LAB SEC: ABL-04

1. AIM

To implement standard list manipulations in Python including computing total sum, average, searching element target, and sorting numerical elements.

2. PROCEDURE

1. Construct a Python list from space-separated input elements parsed as integers.
2. Compute total sum using the built-in `sum()` function and count elements using `len()`.
3. Calculate average value by dividing sum by element count.
4. Prompt for a target value and perform linear search using the `in` operator and `.index()` method.
5. Sort the list in ascending order using `.sort()`.
6. Print computed summary results.

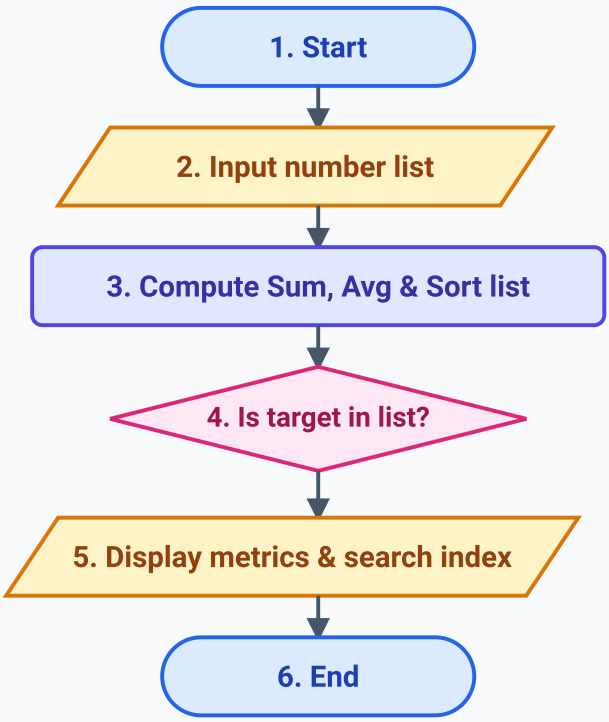
3. ALGORITHM

1. **Step 1:** Start program.
2. **Step 2:** Accept elements into array/list `numbers`.
3. **Step 3:** Compute `total = sum(numbers)` and `avg = total / len(numbers)`.
4. **Step 4:** Read `target` key element for search.
5. **Step 5:** If `target` present in list, retrieve index, else notify missing.
6. **Step 6:** Execute `numbers.sort()`.
7. **Step 7:** Display sorted elements and stats.
8. **Step 8:** Stop.

4. PSEUDO CODE

```
BEGIN
   READ space_separated_string
   CONVERT to LIST numbers
   SET total = SUM(numbers)
   SET avg = total / LENGTH(numbers)
   READ key
   IF key IN numbers THEN
      PRINT "Found at index", INDEX(key)
   ELSE
      PRINT "Not found"
   END IF
   SORT numbers ASCENDING
   PRINT total, avg, numbers
END
```

5. FLOWCHART



6. PROGRAM CODE

```
numbers = [int(x) for x in input("Enter integers separated by space: ").split()]

total_sum = sum(numbers)
avg = total_sum / len(numbers) if numbers else 0

print(f"Total Sum: {total_sum}")
print(f"Average: {avg:.2f}")

key = int(input("Enter element to search: "))
if key in numbers:
    print(f"Element {key} found at index {numbers.index(key)}.")
else:
    print(f"Element {key} not found in list.")

numbers.sort()
print(f"Sorted List: {numbers}")
```

7. OUTPUT

```
Enter integers separated by space: 45 12 89 23 7
Total Sum: 176
Average: 35.20
Enter element to search: 89
Element 89 found at index 2.
Sorted List: [7, 12, 23, 45, 89]
```

8. CONCLUSION

Python dynamic list methods were successfully demonstrated for mathematical calculations, key search, and sorting.

EXPERIMENT 05: Dictionary Character Frequency Counter — LAB SEC: ABL-05

1. AIM

To design a Python script using dictionaries to count the frequency of each unique character in a given text string.

2. PROCEDURE

1. Read a text string input from the user.
2. Instantiate an empty key-value dictionary object `freq_dict`.
3. Iterate over each character in the string.
4. Update character occurrences via `freq_dict[char] = freq_dict.get(char, 0) + 1`.
5. Iterate and format the output key-value map.

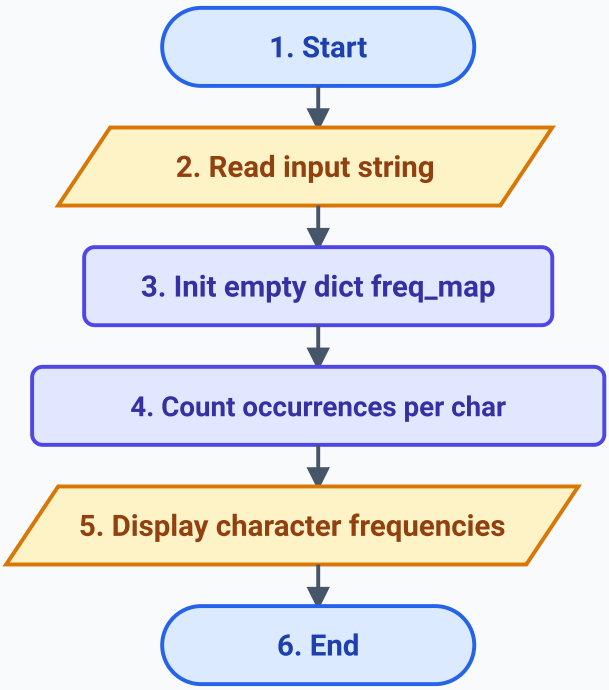
3. ALGORITHM

1. **Step 1:** Start program.
2. **Step 2:** Input string `text`.
3. **Step 3:** Initialize key-value dictionary `freq_map = {}`.
4. **Step 4:** For each character `ch` in `text`: increment `freq_map[ch]`.
5. **Step 5:** Print final character frequency mappings.
6. **Step 6:** Stop.

4. PSEUDO CODE

```
BEGIN
   READ text_str
   INIT dictionary freq_map
   FOR EACH char IN text_str DO
      IF char IN freq_map THEN
         freq_map[char] = freq_map[char] + 1
      ELSE
         freq_map[char] = 1
      END IF
   END FOR
   PRINT freq_map
END
```

5. FLOWCHART



6. PROGRAM CODE

```
input_string = input("Enter a string: ")
freq_dict = {}

for char in input_string:
    freq_dict[char] = freq_dict.get(char, 0) + 1

print("\nCharacter Frequency Map:")
for char, count in freq_dict.items():
    print(f"'{char}': {count}")
```

7. OUTPUT

```
Enter a string: abl python
Character Frequency Map:
'a': 1
'b': 1
'l': 1
' ': 1
'p': 1
'y': 1
't': 1
'h': 1
'o': 1
'n': 1
```

8. CONCLUSION

Efficient character hashing and key-value counting were successfully demonstrated using Python dictionaries.

EXPERIMENT 06: File I/O Operations & Metric Analysis — LAB SEC: ABL-06

1. AIM

To program Python File I/O operations by creating a text file, writing sample text, reading back the file contents, and calculating lines, words, and characters.

2. PROCEDURE

1. Open a file named `sample.txt` in write mode (`'w'`) using the `with open()` context manager.
2. Write multiple sample text lines to the file.
3. Re-open the file in read mode (`'r'`).
4. Read file lines via `readlines()`.
5. Iterate line by line to calculate counts: total lines, split word count, and total character length.
6. Close file context automatically and print calculated text metrics.

3. ALGORITHM

1. **Step 1:** Start program.
2. **Step 2:** Open `sample.txt` in write mode and populate initial content.
3. **Step 3:** Open `sample.txt` in read mode.
4. **Step 4:** Read lines, set `num_lines = len(lines)`.
5. **Step 5:** Compute total words using `split()` and total characters using `len()`.
6. **Step 6:** Print file statistics.
7. **Step 7:** Stop.

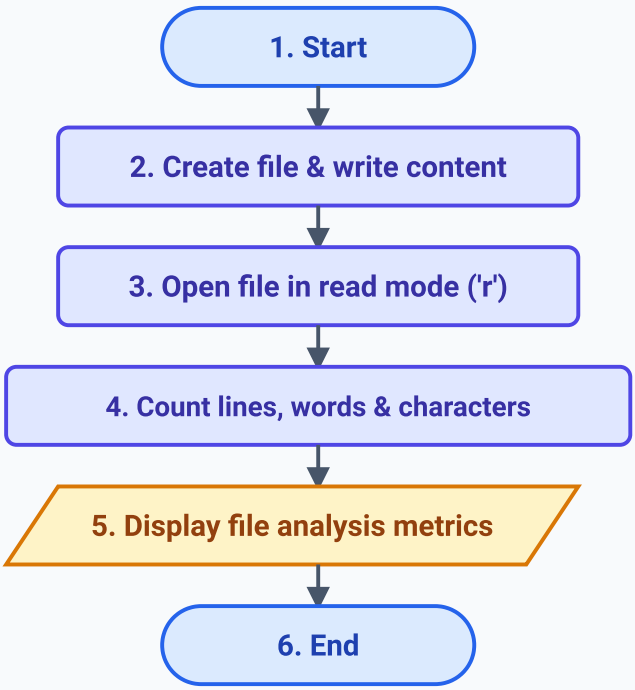
4. PSEUDO CODE

```
BEGIN
   OPEN "data.txt" IN WRITE MODE AS f
   WRITE text lines TO f
   CLOSE f

   OPEN "data.txt" IN READ MODE AS f
   SET lines = READLINES(f)
   SET line_count = LENGTH(lines)
   SET word_count = 0, char_count = 0

   FOR EACH line IN lines DO
      word_count = word_count + LENGTH(SPLIT(line))
      char_count = char_count + LENGTH(line)
   END FOR
   PRINT line_count, word_count, char_count
END
```

5. FLOWCHART



6. PROGRAM CODE

```
# Writing data to file
filename = "sample_lab.txt"
with open(filename, "w") as file:
    file.write("Welcome to ABL Python Laboratory.\n")
    file.write("File handling is a crucial feature.\n")
    file.write("End of sample record.\n")

# Reading and calculating metrics
line_count = 0
word_count = 0
char_count = 0

with open(filename, "r") as file:
    for line in file:
        line_count += 1
        words = line.split()
        word_count += len(words)
        char_count += len(line)

print(f"File Analysis Results for '{filename}':")
print(f"Total Lines: {line_count}")
print(f"Total Words: {word_count}")
print(f"Total Characters: {char_count}")
```

7. OUTPUT

```
File Analysis Results for 'sample_lab.txt':
Total Lines: 3
Total Words: 15
Total Characters: 91
```

8. CONCLUSION

File write, read, and content analysis operations were executed accurately using Python standard file contexts.